

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Requirements Smells: Severity, Frequency and Impact.

Emanuele Gentili

Summary

- Context
- State of the art
- Industrial Characterization of Requirements Smells
- Ticket-Augmented JIT Defect Prediction
- Conclusions

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Context

Requirements Engineering

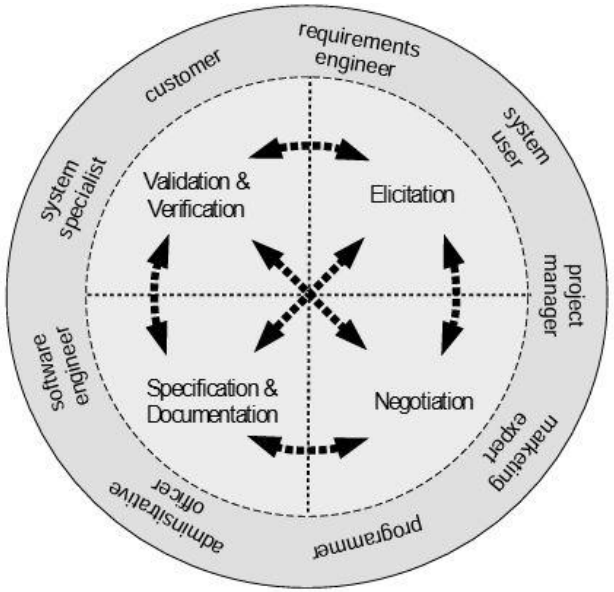


Fig.1 - The four tasks of RE process. [2]

Requirements Engineering (RE)

- «**Systematic**, **iterative** and **disciplined** approach to develop explicit requirements and system specification that all stakeholders agree on» [1]
- Explore and understand the «**Problem Space**» of SW Development project (***why*** and ***what*** to develop) and not the «Solution Space» (*how* to develop) [2]

Main activities

- Elicitation: obtaining **Needs** from Clients/Customers
- Specification & Documentation: improving Needs, creating **Requirements**
- Negotiation: **all stakeholders agree** on the requirements (different views)
- Verification: ensure developed **product meets requirements** (is the system right?)
- Validation: ensure **Requirements reflect** client **needs** (is it the right system?)

[1] Glinz, Martin. "A glossary of requirements engineering terminology." *Standard Glossary of the Certified Professional for Requirements Engineering (CPRE) Studies and Exam, Version* 1 (2011): 56.
[2] Pohl, Klaus. *Requirements engineering: An overview*. Aachen: RWTH, Fachgruppe Informatik, 1996.

Software Engineering Artefacts

A **Software Engineering (SE) Artefact** is

«a work product that is produced, modified or used by a sequence of tasks that have value to a role» [1]

A SE Artefact is characterised by:

- A **semantic** content
- A **syntactic** structure
- A **physical** representation

Requirements, Tickets, Models, Code, Tests are typical SE artefacts.

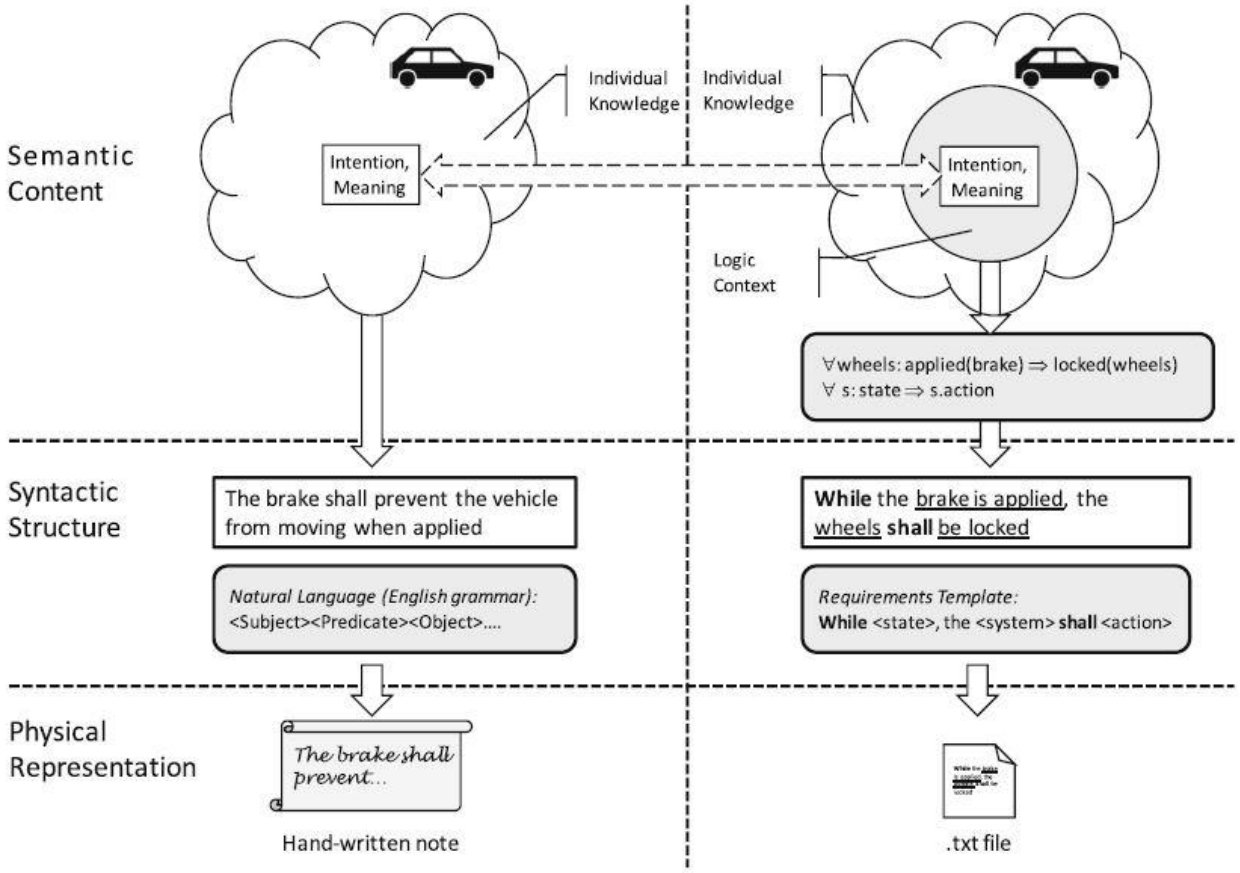


Fig.1 - Example of two artefact with same semantic intention. [3]

[1] Méndez Fernández, Daniel, et al. "Artefacts in software engineering: a fundamental positioning: D. Méndez Fernández et al." *Software & Systems Modeling* 18.5 (2019): 2777-2786..

Requirements Artefacts

Despite several formal methods exists, **Requirements Artefacts** expressed in **Natural Language** (NL) are the most used! Why?

- NL Requirements Arfect have the most understandable and applicable **syntactic structure** [1]
- RE involves a heterogeneous set of **stakeholders** with varying levels of technical **background**

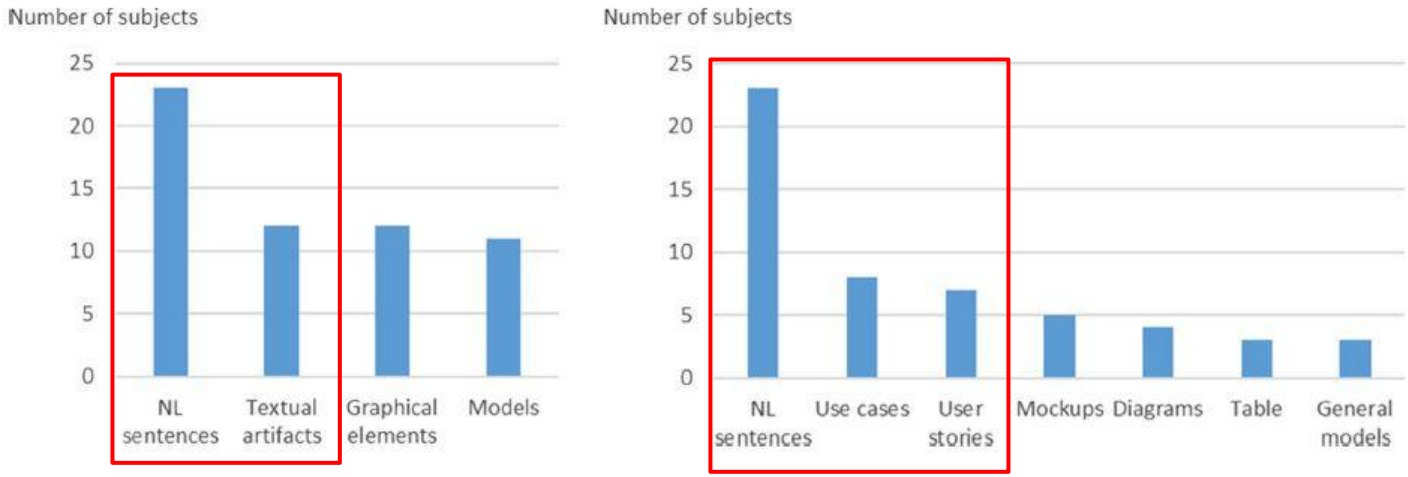


Fig.1 - An overview of the requirements specification artefacts that were used by the interviewees. The graph on the left depicts the main artefact categories that were used, and the graph on the right indicates the frequency with which these artefacts were used [1]

[1] Franch, Xavier, et al. "The state-of-practice in requirements specification: an extended interview study at 12 companies." *Requirements engineering* 28.3 (2023): 377-409.

Requirements Quality

Thus, how can we assess if a given corpus of Requirements is «good»?

Standards specify that a requirement shall be:

- Clear
- Verifiable
- Correct
- Complete
- Feasible
- …



i.e. it shall **possess** all those **characteristics** aiming to **minimise** the «risk» of «misunderstanding» for Practitioners, and consequently minimise the **negative impact** on project activities.

Requirements QA Standards

Requirement Quality Research proposed several «guides» to assess QA:

- *ISO/IEC/IEEE 29148:2018 - Systems and Software Engineering/Life Cycle Processes/Requirements Engineering* [1]
- *INCOSE - Guide for writing requirements* [2]

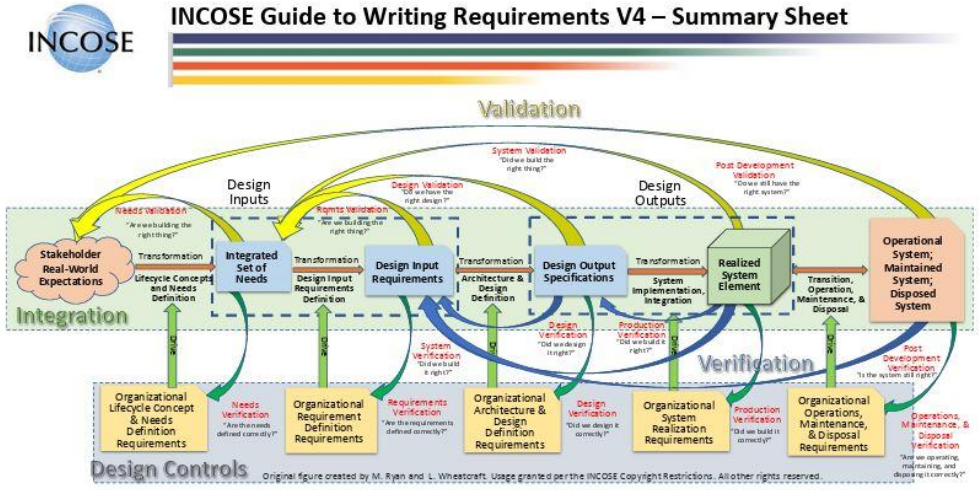

INTERNATIONAL STANDARD

ISO/IEC/ IEEE 29148

Second edition 2018-11

Systems and software engineering — Life cycle processes — Requirements engineering

Ingénierie des systèmes et du logiciel — Processus du cycle de vie — Ingénierie des exigences

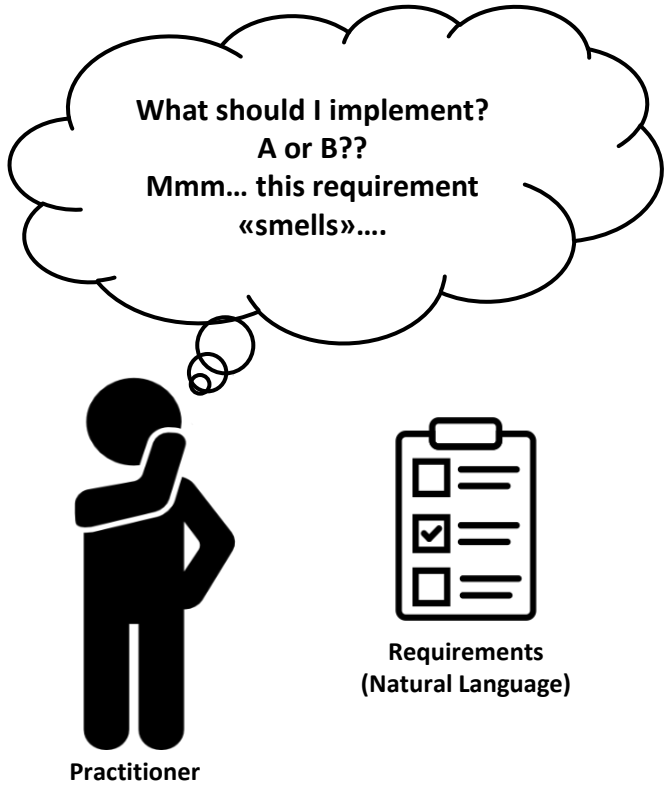


Together, they define **what** a *Software Requirements Specification* (SRS) document must contain (ISO 29148) and **how** to write it without «flaws» (INCOSE) to ensure project success (or better, minimise project failure).

[1] *ISO/IEC/IEEE 29148:2018 - Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*. 2nd ed., ISO/IEC/IEEE, 2018.
[2] INCOSE, Guidance for writing requirements, https://www.incose.org/docs/default-source/working-groups/requirements-wg/guidetowritingrequirements/incose_rwg_gtwr_v4_summary_sheet.pdf

Requirements Smells

What if a Requirement does «meet» the standards' characteristics?



REQUIREMENTS SMELLS [2]

- Ambiguity
- Incompleteness
- Inconsistency
- Incorrectness
- Infeasibility
- Plurality (Non-atomicity)
- Inappropriateness
- Non-Verifiability
- Non-Necessarieness
- Non-Conforming

Migrating the concept of «Code smell», we say that a Requirement is affected by «smell»! [1]

[1] Femmer, Henning, et al. "Rapid quality assurance with requirements smells." *Journal of Systems and Software* 123 (2017): 190-213.
[2] INCOSE, Guidance for writing requirements, https://www.incose.org/docs/default-source/working-groups/requirements-wg/guidetowritingrequirements/incose_rwg_gtwr_v4_summary_sheet.pdf

Requirements Smells - Definitions

A Requirement Smell (RS) represents a **syntactic** or **semantic** quality violation!

Requirements Smells	Definitions [1]
Ambiguous	The statement is complex or vague, allowing for multiple, conflicting interpretations.
Incomplete	The requirement lacks the critical information needed to stand on its own or be fully understood.
Inconsistent	The requirement conflicts with others, uses non-homogeneous units, or introduces contradictory terminology.
Incorrect	The requirement is a "false" representation that doesn't accurately reflect the original stakeholder need.
Infeasible	The requirement cannot be realistically achieved within the given technical, financial, or time constraints.
Plural (Non-singular)	The requirement combines multiple functions or constraints into one, making it hard to manage or verify.
Inappropriate	The level of detail is wrong for the current abstraction layer (e.g., forcing a specific design/architecture too early).
Unverifiable	The wording is not measurable or structured enough to prove its realization to the customer.
Unnecessary	The requirement is obsolete, redundant, or defines a non-essential capability that doesn't fix a real deficiency.
Non-Conforming	The text fails to follow the approved standard template or the prescribed organisational writing style.

[1] Extracted from INCOSE, Guidance for writing requirements, https://www.incose.org/docs/default-source/working-groups/requirements-wg/guidetowritingrequirements/incose_rwg_gtwr_v4_summary_sheet.pdf

Requirements in SDLC

Requirements Artefacts are the **conceptual foundation** upon which:

- Business and Financial plans
- Architecture and Design decisions
- Implementation choices
- Testing strategies and criteria
- Maintenance plans

are built during the Software Development Life Cycle (SDLC)!

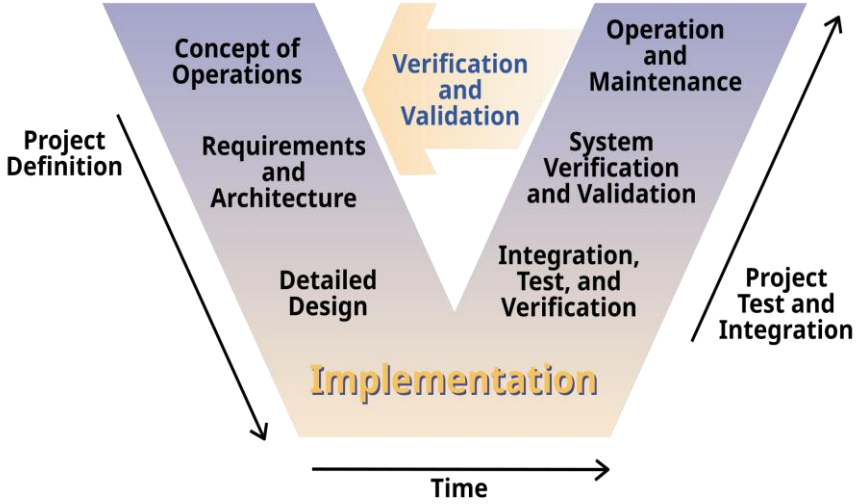


Fig.1 – System Engineering V-Model [Wikipedia]

Requirements Smells can have (potentially) a **negative impact** on overall SDLC activities!

WHAT & HOW?

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

State of the art, Problems, Gaps

Requirements Engineering vs Software Engineering | Research vs Industry

Studies on RE vs Industry

NaPiRE: NAming the Pain in Requirements Engineering [1]

- **Survey** involving 228 worldwide companies
- Analyse problems from literature vs criticality/relevance in different contexts
- Probabilistic problems **cause/effects**

Interestingly:

- Top 4/10 Causes are RE-related
- Top 3/10 Problems are RE-related

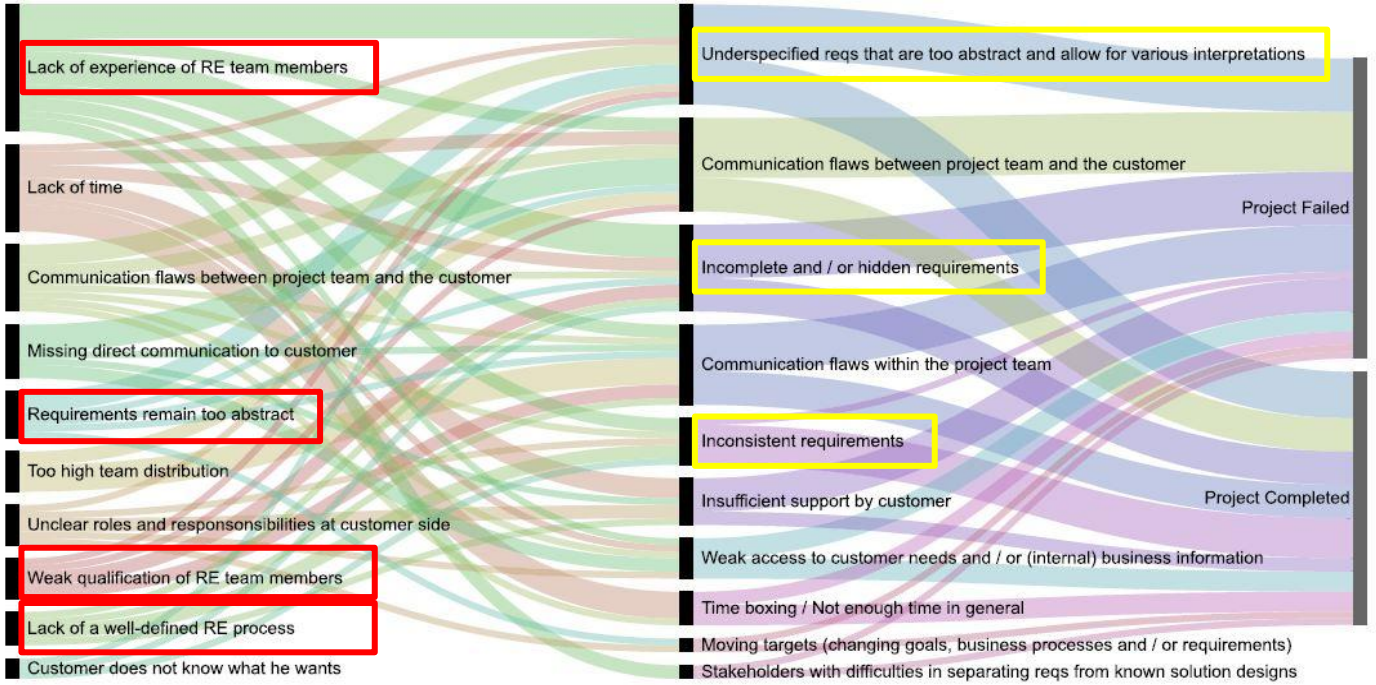


Fig1. Relation of top 10 causes, top 10 problems, and the project impact [1]

[1] Fernández, D. Méndez, et al. "Naming the pain in requirements engineering: Contemporary problems, causes, and effects in practice." *Empirical software engineering* 22.5 (2017): 2298-2338.

Studies on RE vs Industry

NaPiRE: NAming the Pain in Requirements Engineering [1]

- *Incomplete/hidden requirements* effects
  - Project/Organisation (43%)
  - Design and Implementation (43%)
  - Product (21%)
  - Customer (9%)
  - V&V (5%)

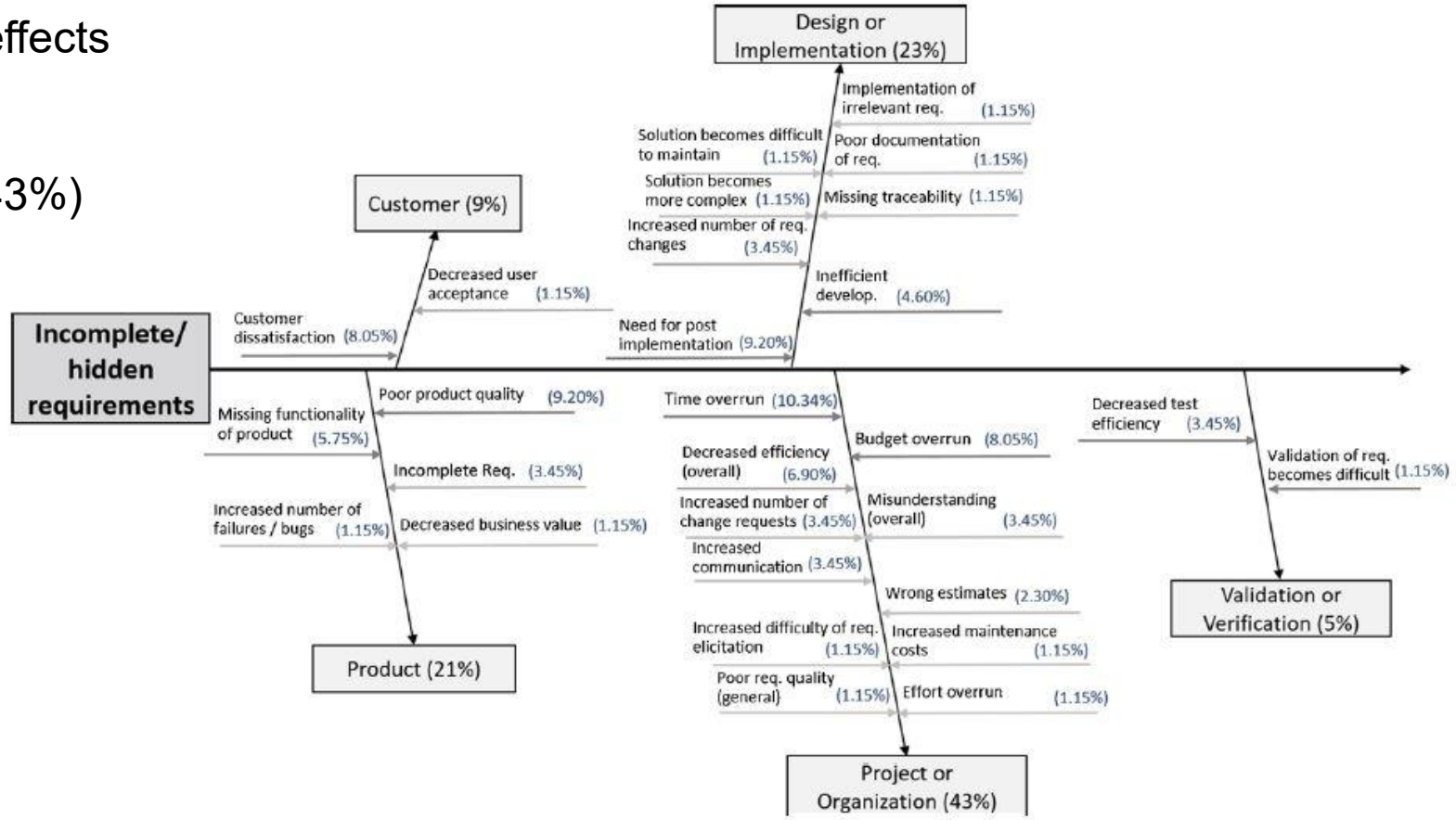


Fig1. Probabilistic cause/effects diagram for Incomplete/Hidden requirements (effects) [1]

[1] Fernández, D. Méndez, et al. "Naming the pain in requirements engineering: Contemporary problems, causes, and effects in practice." *Empirical software engineering* 22.5 (2017): 2298-2338.

Studies on Research on Requirements Quality

A systematic mapping study on requirements quality [1]

- Analyse 105 primary studies
- Extract and Establish 12 «Quality attributes themes»

Ambiguity, Completeness, Complexity, Consistency, Correctness, Redundancy, Relevancy, Reusability, Traceability, Understandability, Undefined, Verifiability

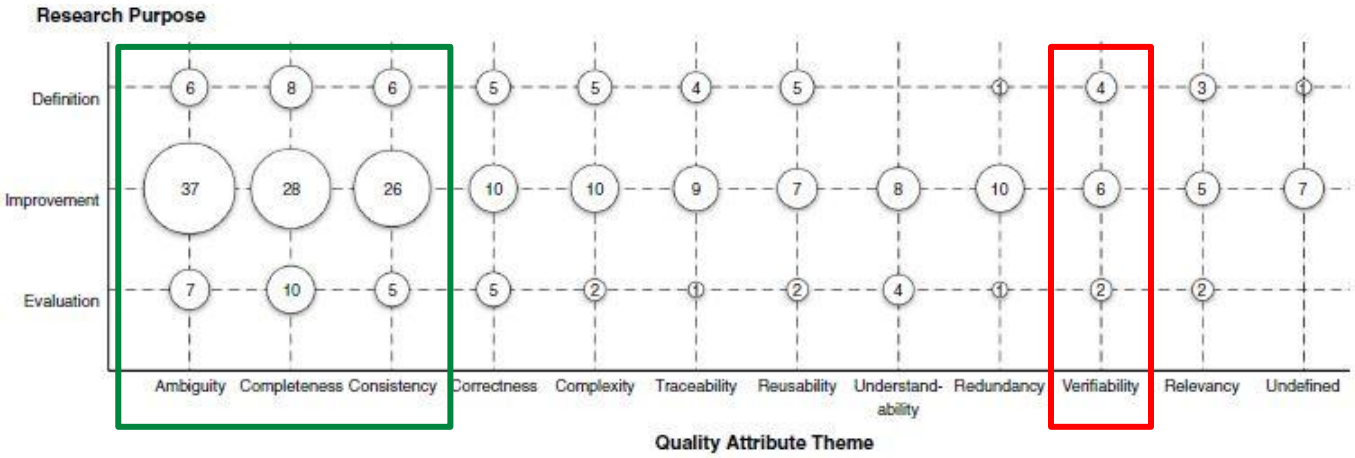


Fig2. Quality attributes mapped against research purpose for each article[1]

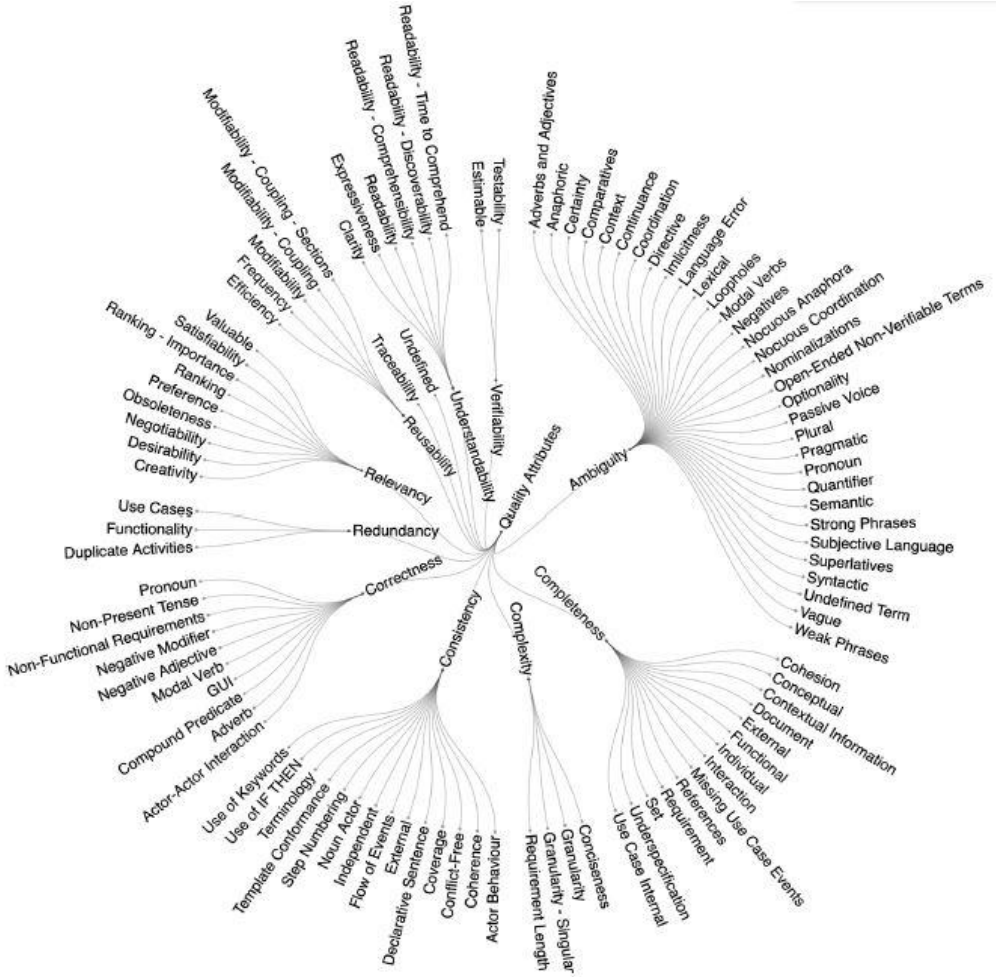


Fig1. Quality attributes “codes” vs quality attributes themes [1]

[1] Montgomery, Lloyd, et al. "Empirical research on requirements quality: a systematic mapping study." *Requirements Engineering* 27.2 (2022): 183-209.

RE Strategies for RS

PREVENTION		DETECTION	
TRADITIONAL	**MODERN**	**TRADITIONAL**	**MODERN**
HAND-WRITTEN **SOLUTIONS** Based on standards - INCOSE - IEC/IEEE/ISO 29148 **LIMITATION** - Based on linguistic patterns - Slow and error-prone - Impact unknown	**SYNTAX/SEMANTIC-ORIENTED** **SOLUTIONS** Tools using Generative AI [3] **LIMITATION** - Interpretability - Controllability - Reproducibility - Hallucinations - Quality of training data - Not mature for Companies	**SYNTAX-ORIENTED** **SOLUTIONS** Automated RS Detection Tools (41) - Smella [1] - Paska [2] - …. **LIMITATION** - Only syntax suggestions - Limited domain (financial) - No guess on semantics - Not available	**SYNTAX/SEMANTIC-ORIENTED** **SOLUTIONS** Tools using NN and DL - BERT - LLM (GPT, …) **LIMITATION** - Data Quality and Availability - Natural Language Complexity - Model Explainability - Domain/Context Adaptation - Integration and Scalability - Traceability Link Quality

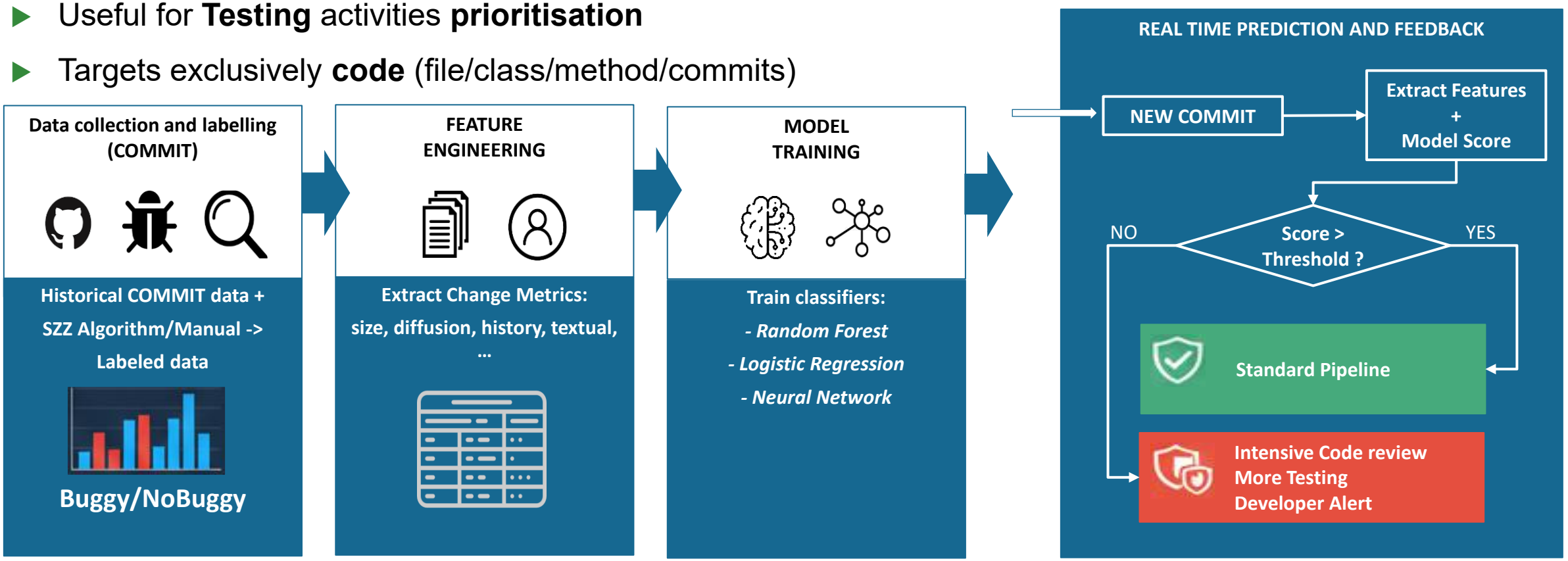
ASSUMPTION: “formally” correct Requirements have “low” impact

[1] Femmer, Henning, et al. "Rapid quality assurance with requirements smells." *Journal of Systems and Software* 123 (2017): 190-213.
[2] Veizaga, Alvaro, Seung Yeob Shin, and Lionel C. Briand. "Automated smell detection and recommendation in natural language requirements." *IEEE Transactions on Software Engineering* 50.4 (2024): 695-720.
[3] Cheng, Haowei, et al. "Generative ai for requirements engineering: A systematic literature review." *Software: Practice and Experience* 56.2 (2026): 141-170.

SE – Traditional JIT Defect Prediction

Traditional Just-In-Time Defect Prediction Models

- Based on Machine-Learning, it predicts the likelihood for a commit to introduce **defects**!
- Useful for **Testing** activities **prioritisation**
- Targets exclusively **code** (file/class/method/commits)



SE – Traditional JIT DP Limitations

Data collection and labelling (COMMIT)



Historical COMMIT data + SZZ Algorithm/Manual -> Labeled data



Buggy/NoBuggy

IMBALANCED DATASET
buggy commits << # clear commits

FEATURE ENGINEERING

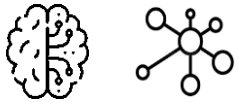


Extract Change Metrics:
size, diffusion, history, textual, …



LIMITED TEMPORAL PREDICTION POWER
Target features “exist” only at late stages, i.e. code

MODEL TRAINING



Train classifiers:
- *Random Forest*
- *Logistic Regression*
- *Neural Network*

MODEL DRIFT
need frequent re-train

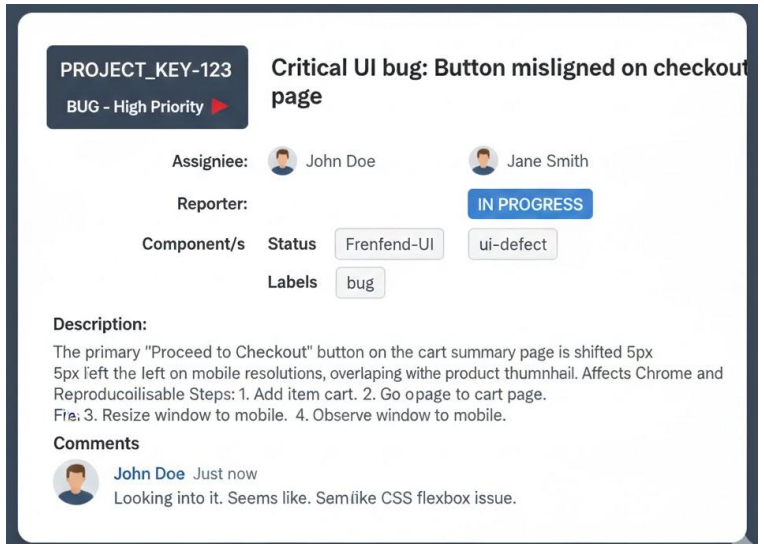
Latest JIT DP approaches

In SDLC, Requirements are usually **operationalised** through **Tickets**:

- Mix functional intents and implementation details
- Very common in Open-source projects
- Not standardised -> difficulty in exploiting its content for Early Defect Prediction [1]



REQUIREMENTS



TICKETS

HIGH-LEVEL ARTEFACT FEATURES [1]

- **Type**
- **Priority**
- **Severity**
- **Some NLP Textual analysis**
- **Developer-centric metrics [2]**
- **…**

LARGELY UNEXPLORED

[1] Özakıncı, Rana, and Ayça Tarhan. "Early software defect prediction: A systematic map and review." *Journal of Systems and Software* 144 (2018): 216-239.
[2] Matsumoto, Shinsuke, et al. "An analysis of developer metrics for fault prediction." *Proceedings of the 6th international conference on predictive models in software engineering*. 2010.

Gaps

REQUIREMENT ENGINEERING

📍 **Relevance:** Which RS matter the most?

📊 **Frequency:** Which RS occur the most?

🔗 **Influence:** What/How RS Impact on SDLC?

SOFTWARE ENGINEERING

💻 **Influence:** What and How RS impact Code?

🔮 **Prediction:** How RS improve Defect Prediction?

⏱️ **Early Action:** How timely can be Defect Prediction?



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Industrial Characterization of Requirements Smells

Practitioners' perception on Requirements Smells

Information and Software Technology 187 (2025) 107823

Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof



Practitioners' perceptions on requirements smells

Emanuele Gentili [a,b], Davide Falessi [b,*]

[a] MBDA Italy S.p.a., Via Monteflavio, 45, Rome, 00132, Lazio, Italy
[b] University of Rome Tor Vergata, Via del Politecnico, 1, Rome, 00132, Lazio, Italy

ARTICLE INFO

Keywords:
Requirement smells
Requirement quality
Industrial case study

ABSTRACT

Context: Software specifications are usually written in natural language and may suffer from imprecision, ambiguity, and other quality issues, hereafter referred to as requirement smells. Requirement smells can hinder project development in many aspects, such as delays, reworks, and low customer satisfaction. From an industrial perspective, we want to focus our time and effort on identifying and preventing the requirement smells of high interest. We also want to identify the metrics to measure the effect of smells on a software project.

Objective: We aim to characterize types of requirement smells in terms of frequency, severity, and effects. To the best of our knowledge, no previous study analysed how frequency, severity, or effects vary across types of smells.

- https://doi.org/10.1016/j.infsof.2025.107823

Aim

Aim

From an industrial perspective, we want to focus our time and effort on **identifying** and **preventing** the Requirements Smells that are **important**.

- Characterize Requirements Smells (Natural Language) in terms of **Severity**, **Frequency** and **Effects** on SDLC

Why?

- Removing all the requirements smells is **expensive**, especially for the high impact they can have on several artefacts.
- Knowing which smell is **important** for **whom**, **when** and **why** reasonably supports the **reduction** and **prevention** of smells.

Research Questions

- **RQ1**: What are the most **severe** RS, as perceived by practitioners?
- **RQ2**: What are the most **frequent** RS, as perceived by practitioners?
- **RQ3**: What are the **impacts** of RS on SDLC artefacts, as perceived by practitioners?

Methodology

Empirical Investigation: Interview (Exploratory) + Survey (Confirmatory)

Company: MBDA Italy S.p.a. @ Rome, SE Department

Domain: Mission & Safety Critical

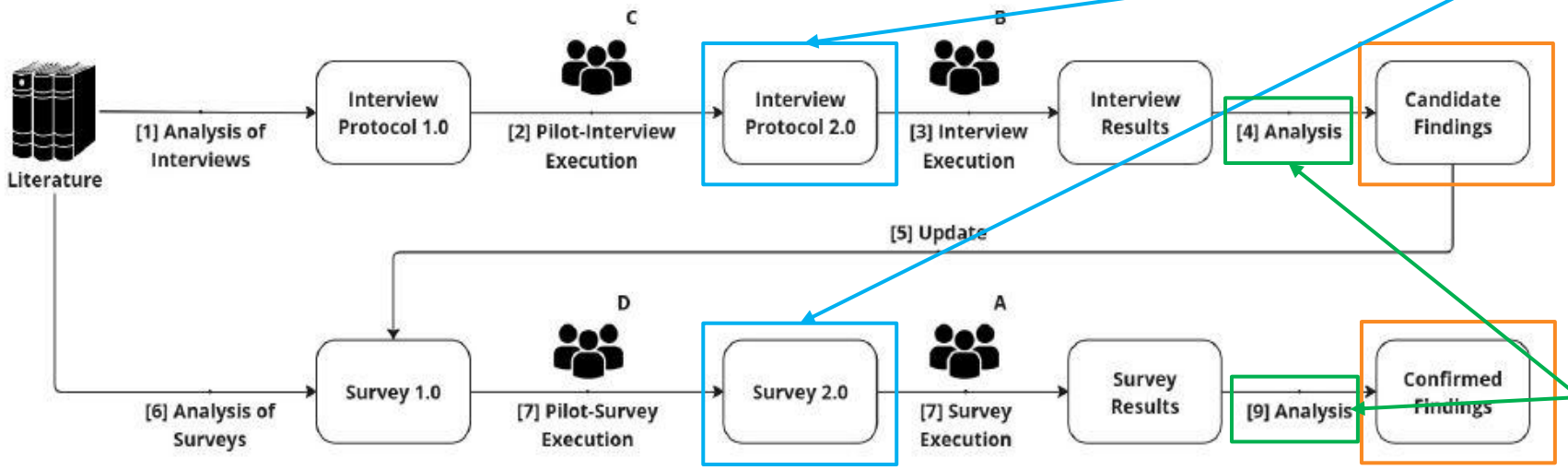


Fig.1 Phase I - Methodology overview – Activities (arrows) and artefacts (boxes)

INVESTIGATION TECHNIQUES

Interview (Semi-structured)
- Open Questions
- Top 3 - Bottom 3 (TB3)

Survey
- 100 Points (100P)
- Likert Scale (LKS)

DATA ANALYSIS

Interview
- Frequency Distributions

Survey
- Frequency Distributions
- Rank-Based Linear Mixed Models
- Wilcoxon Signed Rank Test (*p*-value)
- Bubble Plots
- Multiple Correspondence Analysis (MCA)

[1] William Schofield. 2006. Survey sampling. Data collection and analysis, 2, 332.
[2] Elissa M Redmiles, Yasemin Acar, Sascha Fahl, and Michelle L Mazurek. 2017. A summary of survey methodology best practices for security and privacy researchers.
[3] Ian Brace. 2018. Questionnaire design: How to plan, structure and write survey material for effective market research. Kogan Page Publishers.

Methodology - Interview vs Survey

INTERVIEW

Interviewee characterization

- *Role, #YE (MC/SC)*

Project characterization

- *#Req, # Dev, SRT/HRT, #Exec, #LOC, #YP (SC)*

Requirements Smells characterization

1. *What are the three most and least **severe** RS? (TB3)*
2. *What are the three most and least **frequent** RS? (TB3)*
3. *What are the **effects** of a certain RS? (Open)*
4. *Are there **contexts** in which the **effects** of a certain RS are **mitigated/amplified**? (Open)*
5. *What are the **metrics** that could be used to measure the effects that a certain RS could induce on project development? (Open)*

RS

FINDINGS

METRICS

SURVEY

Demographics

- *Role, #YE, Depart., SRT/HRT, Type of Reqs (MC/SC)*

Agreement with Findings

- *F1: The perceived severity of a smell varies across types of smells. (100P)*
- *F2: The perceived severity of the same smell varies across project domains. (LKS)*
- *…*

Smell-to-Effect linking

Given the list of 13 Metrics (#Defect, #Queries, #Commit, …)

- *For each RS, what metrics could measure the negative impact? (Bubble Plot, MCA)*

Methodology - Data Analysis – Statistical Tests for 100P

Phase	Methodological Challenge (Data Nature)	Statistical Solution	Rationale
1. Global Analysis *(RBLMM - Data Layer)*	**Non -Normality** • Points are *constrained* (sum 100). • Non-normal distribution and skewness	**Rank Transformation**	• Converts raw scores into ordinal ranks (mitigates the impact of extreme outliers). • Shifts focus to relative prioritisation.
2. Global Analysis *(RBLMM – Structure Layer)*	**Subjectivity & Dependancy** • Data points are interdependent (repeated-measures). • High risk of inter-rater variability (respondents possess varying severity baselines)	**Linear Mixed Model**	• Incorporates respondent *ID*s as a *random effect (*controls for individual evaluation biases). • Isolates the *fixed effect* (Smell Type) (objectively determines smell category influence).
3. Post-Hoc Analysis *(WSRT)*	**Pairwise Identification** • Global test indicates an effect but lacks specificity. • Need to identify exactly *which* smell pairs differ. • Requires a robust test for dependent/paired data.	**Non-Parametric Paired Test**	• Evaluates median differences between paired observations. • Enables rigorous, head-to-head comparisons. • Functions without assuming a normal data distribution. • Strictly controls Type I error during multiple tests.

How do we assess the **statistical relevance** of our data (i.e. if a smell is “actually” more severe/frequent than another)?

OMBIBUS (GLOBAL) TEST
Rank-Based Linear Mixed Models (alpha=0.05)

“Is there at least one smell that is significantly different from the others?”

POST-HOC TEST
Wilcoxon Signed-Rank Test (alpha=0.01)

“Exactly which smells are different from each other”?

Methodology - Data Analysis – MCA for RS-to-Metrics

To analyse Smells-to-Effect Linkage, we apply *Multiple Correspondence Analysis (MCA).*

Goal

- Analyse and represent relationships and patterns between categorical variables in a 2D space
- Transforms complex frequency tables into an “intuitive” 2D map
- Proven “beneficial” with *few variables* (<=3) that have *many categories*

How does it work?

📍 Proximity = Association: The closer two points are, the stronger their relationship

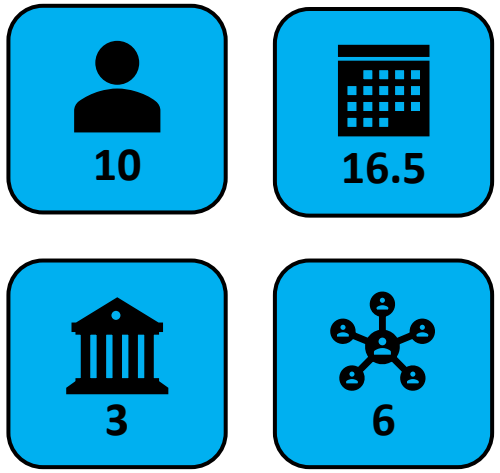
↔ Dimensions = Variance

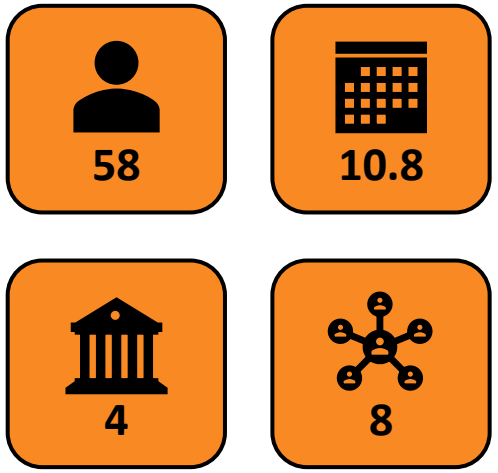
- X-axis, largest variance – capture dominant pattern
- Y-axis, second largest variance – capture subtler differences

🎯 Clustering = Low Impact: The closer categories of a single variable, the less distinguishable the effects of the variable

Results – Demography

INTERVIEW





SURVEY

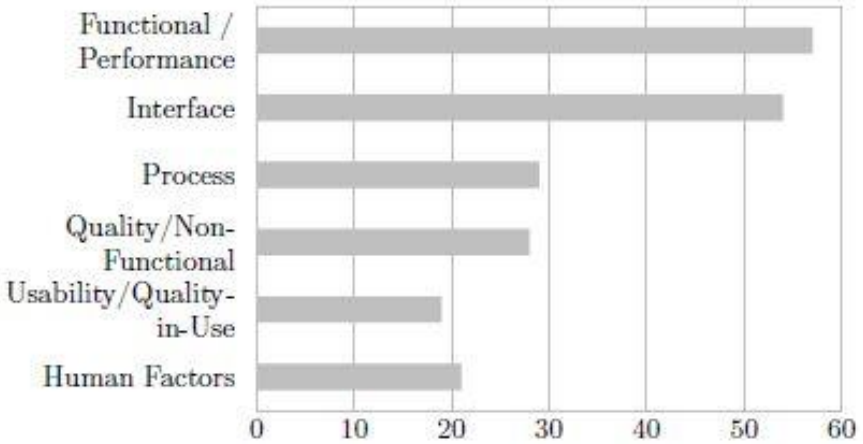


Fig1. Distribution of types of requirements that participants have worked with.

GOOD VARIABILITY OF THE POPULATION

Results – Findings

FINDING 1 RQ1

The perceived severity **varies across types of smells.**

FINDING 2 RQ1

The perceived severity of the smell **varies across project domains.**

FINDING 5 RQ1

The severity of a smell might **change across roles and phases.**

FINDING 3 RQ2

The perceived frequency **varies across types of smells**.

FINDING 4 RQ2

The frequency of a smell is perceived **differently across roles or phases.**

FINDING 6 RQ3

The perceived effects may **vary across types of smells**.

Results – Metrics

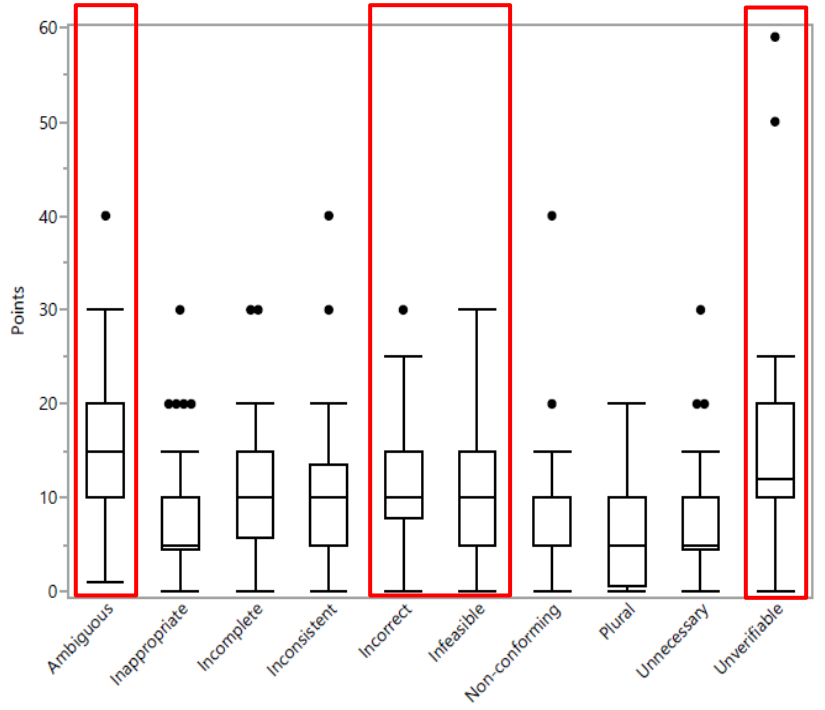
ID	Metric Description
M1	High number of Defects - i.e. bugs during the software development phase
M2	High number of Queries - i.e. requests of clarification about requirements content
M3	High number of Change Requests of the TRS (Technical Requirement Specification) document
M4	High number of code Commits
M5	High number of LOC (lines of code) tested by inspection
M6	High number of Line of Code impacted by Safety Critical standards
M7	Low score for code Quality Assurance - e.g. the presence of duplicate code, high cyclomatic complexity, deep nesting, long methods, inappropriate comments, etc...
M8	High number of Technical Facts - i.e. software bugs in production, coming after a real deployment of a system on the field
M9	High number of document updates - i.e. any documents produced during the SW development lifecycle
M10	High number of test updates/reworks
M11	Delay in SRS (Software Requirement Specification) document definition
M12	Delay in starting coding activities
M13	Others - to let the participants specify other metrics or better specify their choices

Results – RQ1

RQ1: What are the most severe RS, as perceived by practitioners?

- **F1: The perceived severity varies across types of smells**
  - [I/S] *Ambiguity*, *Unverifiability*, *Incorrectness* and *Infeasibility* most Severe!
  - [S] 31/45 combinations of smells are statistically different
- **F2: The perceived severity of the smell varies across project domains**
  - [I] *Unverifiability* in Hard-Real-Time context might be catastrophic!
  - [S] 36% strongly agreed, 59% agreed
- **F5: The severity of a smell might change across roles and phases**
  - [I] Not all the practitioners experience the same "degree" of severity

    E.g. *Incompleteness* is perceived as:
    - **highly severe** from SW Group/Project-Leaders (blocker)
    - **variably severe** from Practitioners:
      - ACCEPTABLE at EARLY stage, since they have room for "experimenting" other solutions
      - NOT ACCEPTABLE at LATE stage (high pressure)
  - [S] 19% strongly agree, 53% agree



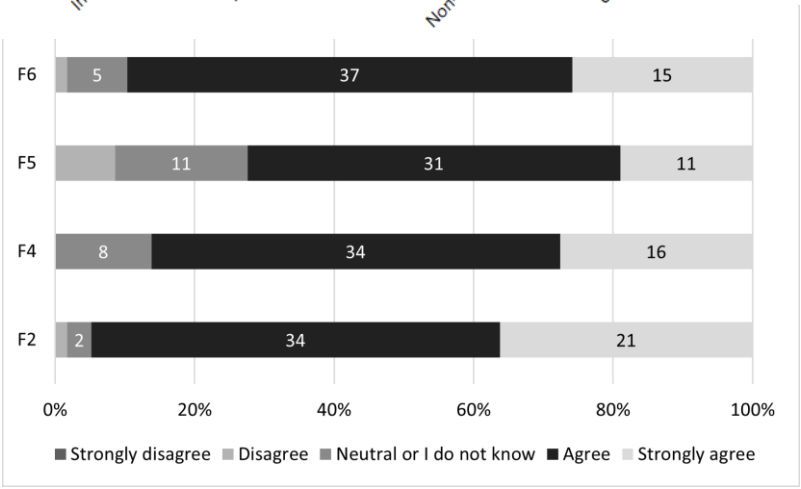


Fig1. Perceived Severity on Requirements Smells
Fig2. Agreement with Findings

Results – RQ2

RQ2: What are the most frequent RS, as perceived by practitioners?

- **F3: The perceived frequency varies across types of smells.**
  - [I/S] *Ambiguity*, *Unverifiability*, *Incompleteness* most Frequent!
  - [S] 34/45 combinations of smells are statistically different
- **F4:The perceived frequency of a smell varies across roles or phases.**
  - [I] Early pre-filtering by experts acts as a shield, reducing the propagation of smells to downstream phases
  - [S] 28% strongly agreed, 59% agreed

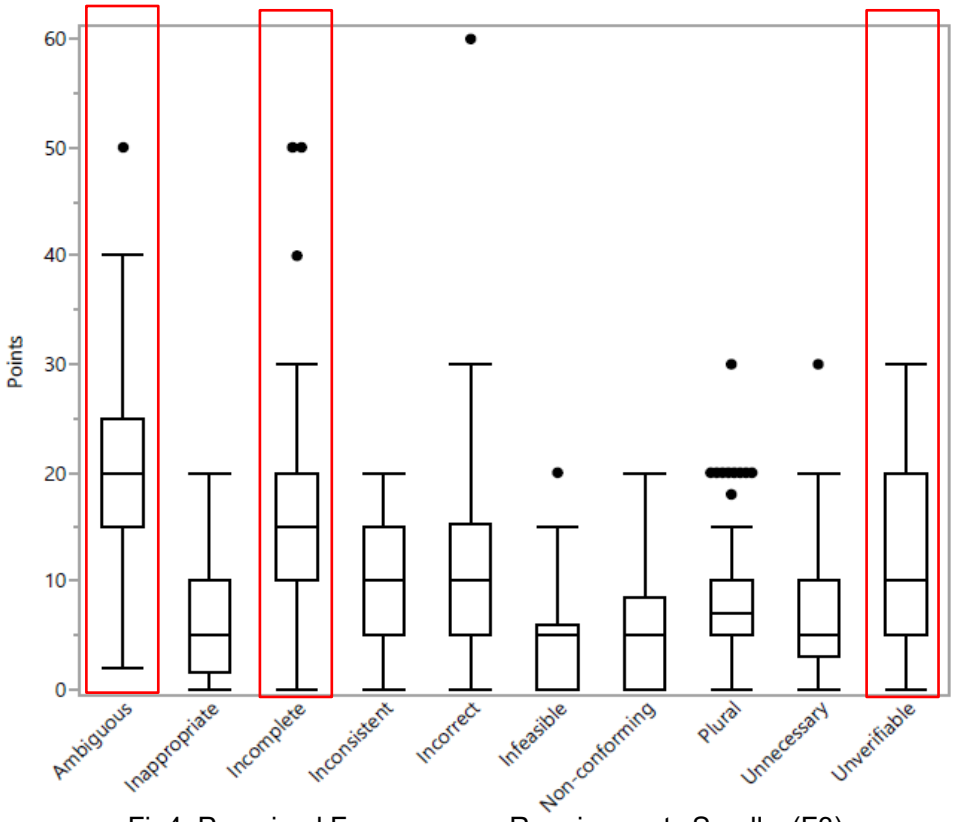


Fig4. Perceived Frequency on Requirements Smells. (F3)

Results – RQ3

RQ3: What are the impacts of RS on SDLC artefacts, as perceived by practitioners?

- **F6: The perceived effects may vary across types of smells**.
  - [I] *Unverifiability*
    - Increased Coding effort (“how do I write code so that I can provide evidence to the customer?”)
    - Increased Testing complexity (“given the code, which test is appropriate to verify the requirement?“)
    - Bug in Production -> low customer satisfaction/loss of credibility -> rework -> extra costs
    - Verification might be incorrect -> bug not found during Integration -> multiple runs (time and cost increase)
  - [I] *Ambiguity*
    - Requirements not clear -> trigger a continuous cycle of “change requests” (RE vs SE)
      - If detected “early” -> little rework
      - If detected “late” (Integration/Production) -> bugs, whose resolution might have HIGH impact on all activities (time/costs)
  - [I] *Complexity*
    - Typically, complex requirements drive Practitioners to unnecessarily “*complex implementation”.* Is this “complexity” needed?
      - Yes: Inherent complexity -> careful! Engage “expert” practitioners for coding/testing (early management)
      - No: simplify the requirement! -> might “trigger” similar problems to Ambiguity/Unverifiability
  - [S] 26% strongly agreed, 64 % agreed

Results – RQ3

Smell-to-Metric Linkage

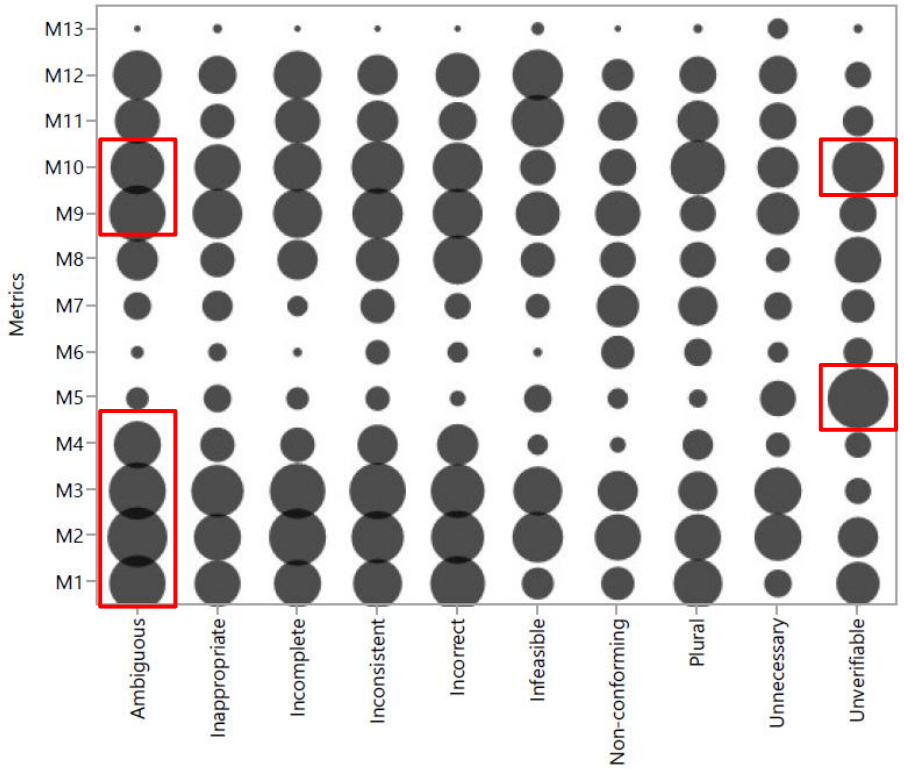


Fig.1. Frequency of Metrics perceived as useful for measuring effects of RS.

M1: High number of Defects
M2: High number of Queries
M3: High number of Change Requests of TRS
M4: High number of Commit
M5: High number of LOC by inspection
M6: High number of LOC impacted by Safety
M8: High number of Technical Facts
M9: High number of document updates
M10: High number of test updates/rework
M11: Delay in SRS definition
M12: Delay in starting coding activities

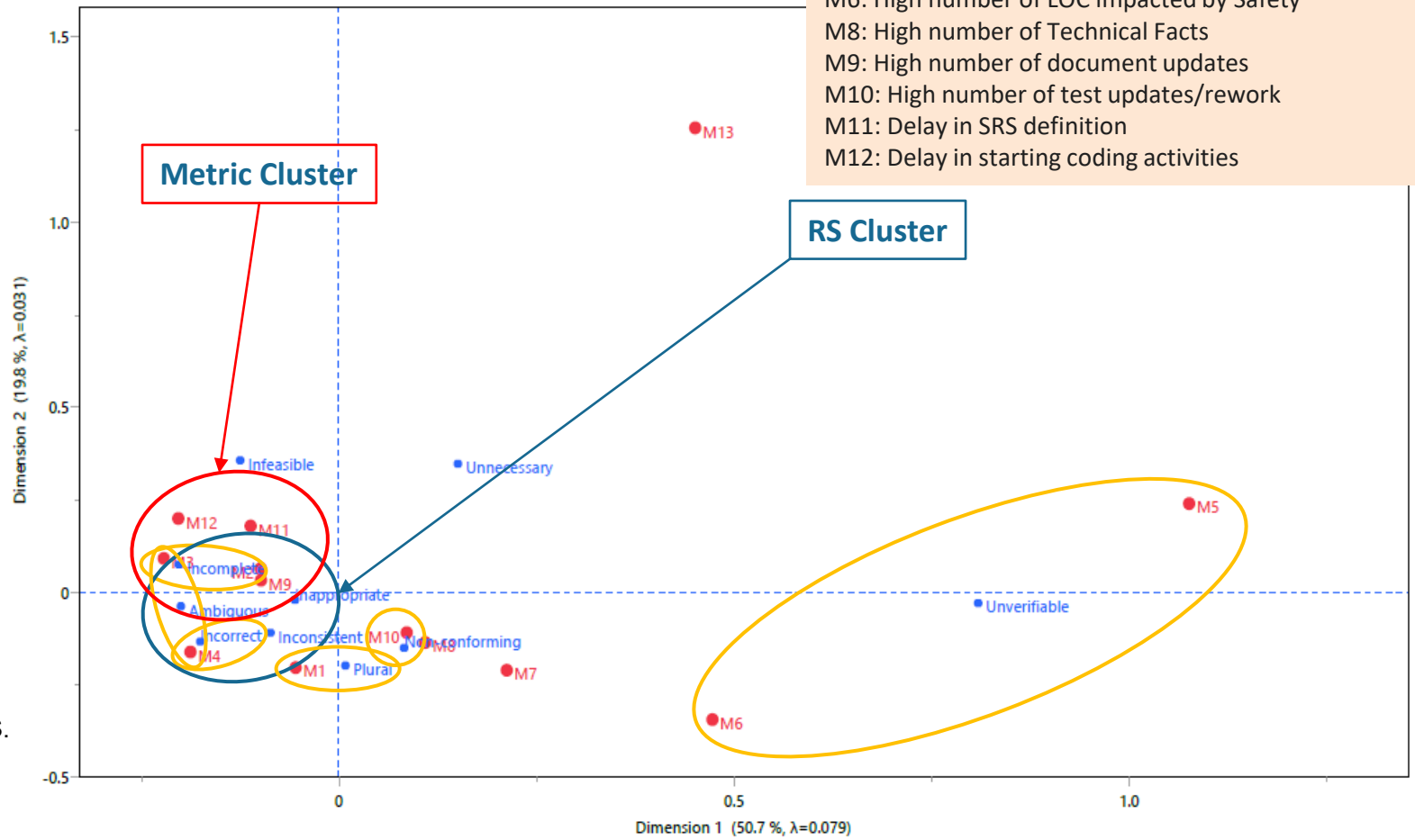


Fig.2. MCA between RS and Metrics for measuring their effects.

Discussion – Severity, Frequency and Context

RS represents real Threats:

- ***Ambiguity*** is perceived as the **most severe** and **most frequent** smell -> primary driver of project friction!
- ***Unverifiability*** is perceived as the second most **severe** smell -> Literature identifies it as historically **understudied**! [1]
- ***Incompleteness*** is perceived as severe and highly frequent -> might have an disruptive effect on architecture if not appropriately managed!

RS effects depend on Context (Domains, Phases & Roles):

- **Domain Amplification (F2):** In Safety-Critical systems, minor smell deviations escalate into severe operational risks!
  - In business apps, minor smells merely cause delays.
- **Roles and Phase Amplification (F5):** Smells like *Incomplete* severely impact downstream software architecture/design, especially in late-stage.
  - SW Group/Project/Team Leaders perceive *Incompleteness* as highly severe (since the very first moment).
  - SW Practitioners like to “experiment” -> perceived as less severe in the early stage.
- **Roles Mitigation (F4):** Smell frequency is not static.
  - Requirement Experts act as a "shield“ for *Infeasible* Requirements, resolving smells before downstream roles (developers) even perceive them.
  - Senior SW Architect & Open/Robust Architectures can mitigate the effect of *Incomplete* Requirements.

[1] Montgomery, Lloyd, et al. "Empirical research on requirements quality: a systematic mapping study." *Requirements Engineering* 27.2 (2022): 183-209.

Discussion – From RS to SDLC Metrics

The Central RS Cluster:

- **Smells:** *Ambiguous, Incomplete, Incorrect, Inconsistent*.
- **Impact:** These “central” smells drive generalised project friction. They strongly **correlate** with high defect rates (M1), excessive code commits (M4), and continuous document/test reworks (M9, M10).

Specific Smells trigger Specific “Pain” (Outliers):

- *Unverifiable*
  - “Manual Effort”: Associated with a *high #LOC tested by manual inspection* (M5).
  - “Safety risks”: Associated with a *high #LOC impacted by Safety Critical standards* (M6).

Actionable Takeaway for Practitioners:

- Practitioner can move from theoretical detection to **symptom-based mitigation.** If the project shows certain effects, they can track back to the most correlated RS.

Conclusions

- **Focus on Important Smells:**
  - Prioritise identifying and preventing **high-impact** Requirement Smells (e.g., *Ambiguity*, *Unverifiability*).
  - Tailored mitigation strategies are essential based on **smell types** and **project stages**.
- **Impact of Early Detection:**
  - Addressing Requirements Smells early **reduces** costly **rework** and **bugs** in later stages.
  - Early detection and targeted monitoring enhance project **stability** and **quality**.
- **Smell Classification:**
  - Classify smells by their impact (e.g., rework, bugs, usability issues) to design specific mitigation strategies.
  - Understanding smell severity and frequency (e.g., Ambiguity as both severe and frequent) informs effective interventions.
- **Complexity Consideration:**
  - Smell reduction does not eliminate inherent complexity in development.
  - Simplifying descriptions may ease development, but complexity remains a challenge.

[1] Montgomery, Lloyd, et al. "Empirical research on requirements quality: a systematic mapping study." *Requirements Engineering* 27.2 (2022): 183-209.



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Ticket-Augmented Just-In-Time Defect Prediction

Context and Aim

Conference: *International Conference on Product-Focused Software Process Improvement (PROFES) 2025*

Authors:

- *Emanuele Gentili, University of Rome Tor Vergata*
- *Daniele La Prova, University of Rome Tor Vergata*
- *Davide Falessi , University of Rome Tor Vergata*

Aim:

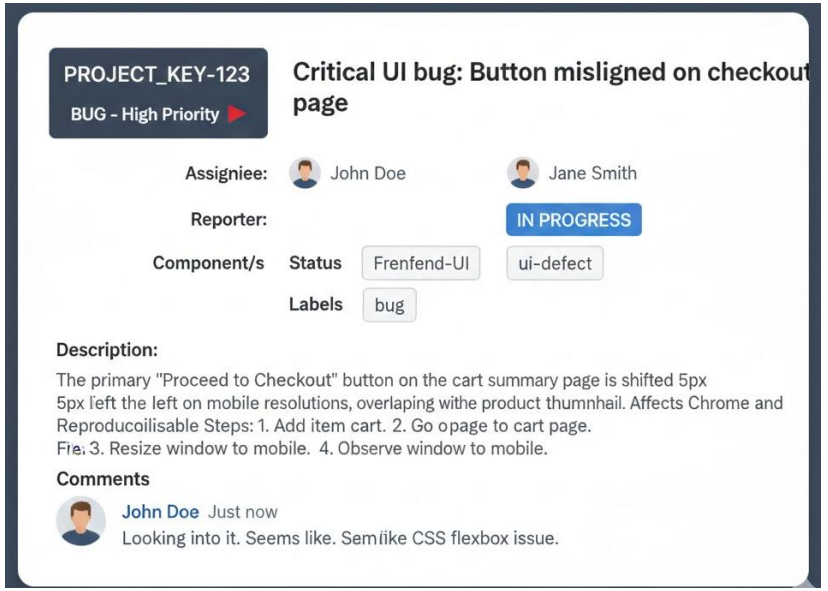
- Use **Ticket-Level information** to **improve accuracy** of standard **JIT** defect prediction models
- Propose and validate a novel model *Ticket-Augmented Just-In-Time Defect Prediction*

Research questions:

- **RQ4**: Does leveraging ticket-level information increase the accuracy of Just-In-Time defect prediction?
- **RQ5**: What is the predictive power of ticket-level features in Just-In-Time defect prediction?

Ticket Properties vs Requirement

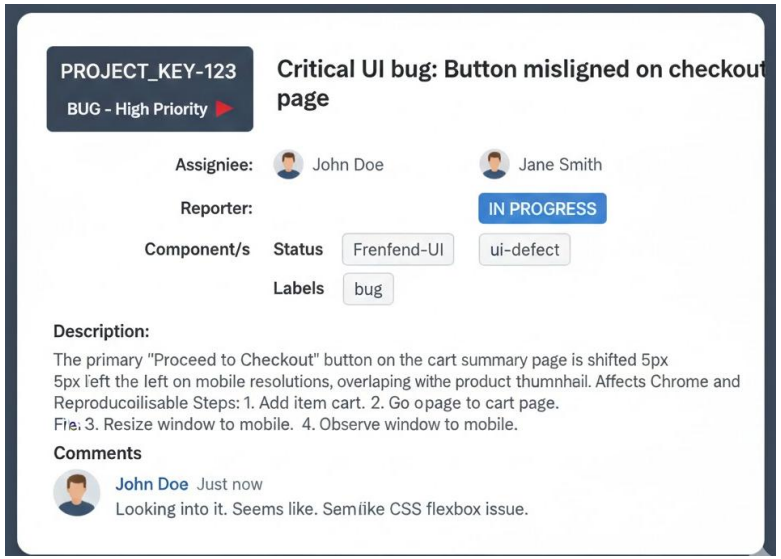
- A **Ticket** is the primary mechanism for **translating** abstract **Software Requirements** into **actionable** and **trackable** units of work.
- Ticket (aka ***Issue*** or *Work item)* is a digital representation of a **task**, **problem**, **request**, **bug** or any unit of work that needs to be tracked and managed within a project.
- Ticket has several **properties**:
  - ID, Title and Description
  - Type, Priority
  - Due date
  - Status
  - Assignee
  - Comments
  - Attachments, Custom-Labels, …



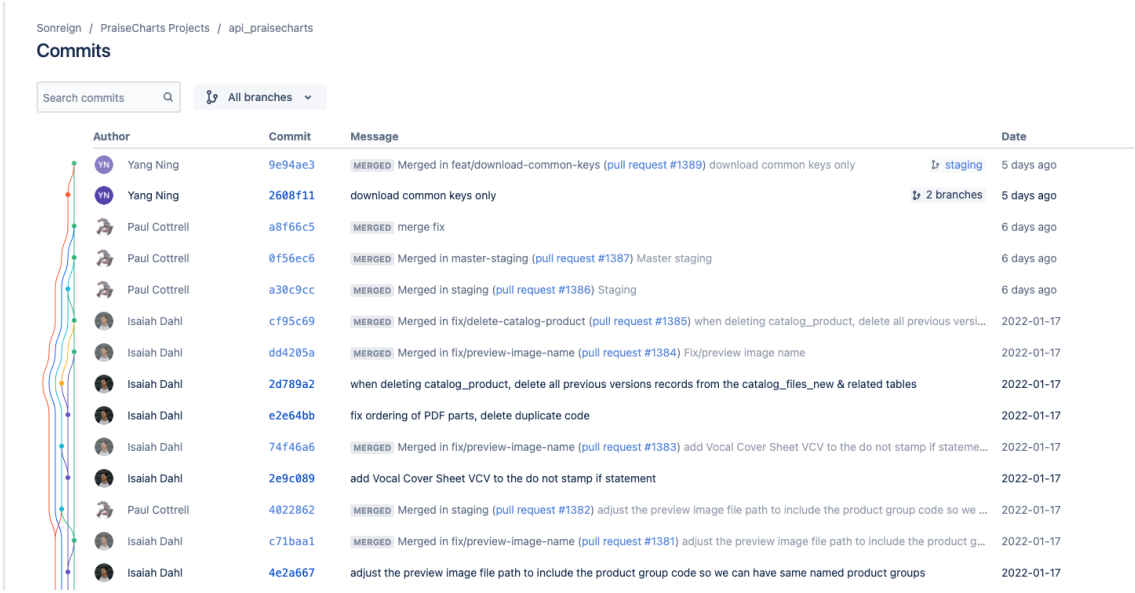
- **Title** and **Description** are usually employed for tracking back **Ticket** to ***Software Requirement***.

TA-JIT Core concept

- We acknowledge the success of traditional Just-in-Time (JIT) Defect Prediction (commit-based)
- But… **COMMITS** implement **TICKETS**, i.e. commits derive from information contained in tickets



TICKET



COMMIT(S)

- So, why not use **information from Ticket** to improve Standard JIT DP performance?

Methodology - TA-JIT Features



TICKET(s)



COMMIT



CODEBASE

[53] Ticket-Level Features* (5 families)

- Developers [D]
- External Temperature [E_T]
- Internal Temperature [I_T]
- Intrinsic [I]
- Ticket-2-Ticket [T2T]

+

[1] Number of aggregated Tickets

[13] Standard JIT Features**

[4] Codebase Features*

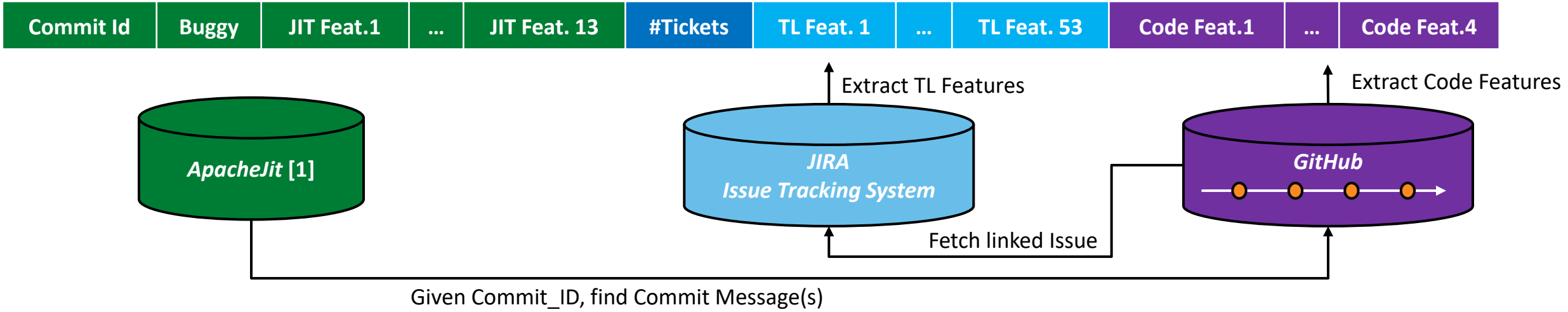
- N° Code Smells
- N° languages
- N° Files
- Total LOCs

[71] TA-JIT Features

Methodology - Ticket-Level Features

- **Developer**: assess the **historical performance** of a developer, with the underlying assumption that a developer's experience and familiarity with a project's codebase—often measured by the number of tickets previously assigned to them—can indicate a lower likelihood of introducing defects.
- **Intrinsic**: capture the **inherent complexity** of a ticket, independent of the code or developer. This includes factors like the number of requirements, ticket type (e.g., bug or feature), and priority level, all of which are assumed to correlate with the likelihood of introducing defects.
- **Internal Temperature**: capture the "hotness" of a ticket by measuring collaboration and discussion, such as the number of comments, and incorporate **sentiment analysis** to identify potential bugs based on negative or uncertain language.
- **External Temperature**: capture the project-wide activity that may interfere with development quality, like the number of open tickets (used as proxy for workload and potential distraction)
- **Ticket-to-Ticket Similarity**: capture the semantic similarity between a given ticket and past tickets that introduced bugs, using established Natural Language Processing (NLP) techniques like TF-IDF, Jaccard similarity, and Euclidean distance on both the title and description to identify similarities that may indicate a ticket is more likely to be bug-prone.

Methodology - Dataset creation

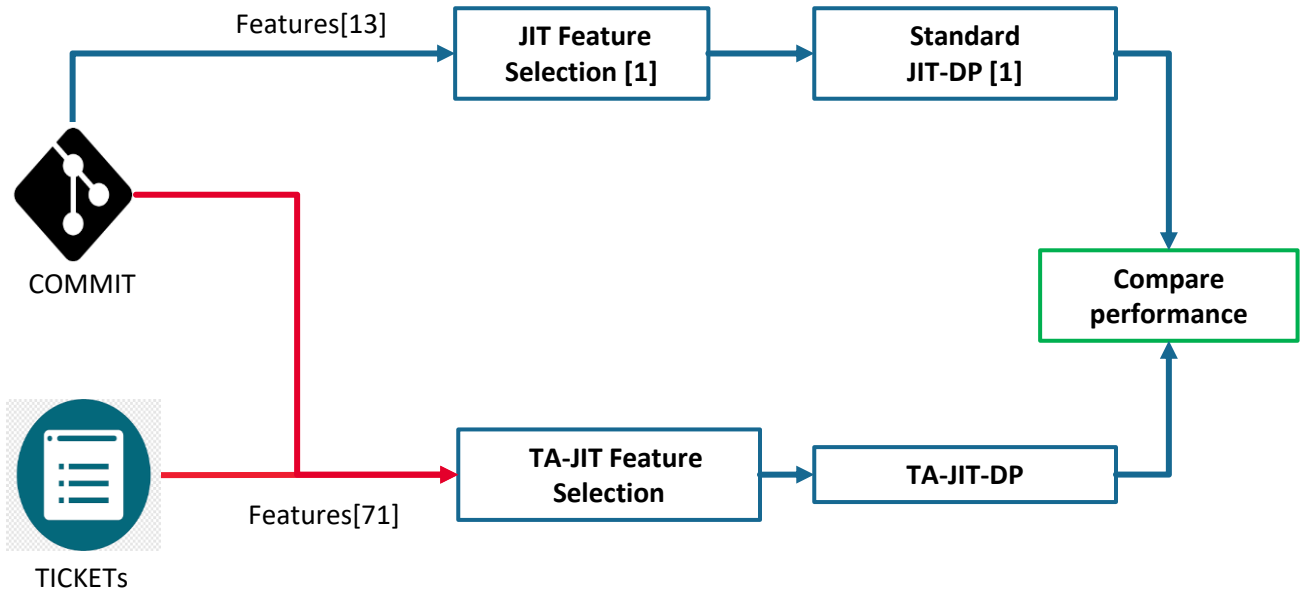


- Problem: Linkage Issue_ID vs Commit_ID (>90%)
  - Identified 2 Projects: *HIVE* and *HBASE* -> approx. 10000 tickets
- Dataset ordered by commit datetime
  - Cannot use Tickets with creation_time > Commit creation time (Temporal inversion)

[1] Keshavarz, Hossein, and Meiyappan Nagappan. "Apachejit: a large dataset for just-in-time defect prediction." *Proceedings of the 19th international conference on mining software repositories*. 2022.

Methodology – JIT vs TA-JIT

- Compare the accuracy of JIT vs TA-JIT



EXPERIMENTAL SETUP

CLASSIFIERS
- Random-Forest (RF)
- Logistic-Regression (LR)
- Feed-forward Neural Network (FFNN)

BALANCED SLIDING WINDOW:
- Window -> 1000 commits
- Delta -> 200 commits (for each iteration)

FEATURE SELECTION:
- **Simmetrical-Uncertainty filter**
  - quickly select features that are "individually" relevant
- **Correlation-based Feature Selection** (CFS) wrapper
  - find the most effective combination of features (by removing weaker ones) to retain highest correlation.

METRICS
- AUC, F1, Precision, Recall, Kappa, G-Mean, G-specificity

DATA ANALYSIS
- Measure **Selection Frequency** of Feature families
- *Wilcoxon Signed-Rank Test (WSRT)*
  - Null Hypothesis: the accuracy of commit-level prediction does not change when augmenting JIT with TL Features
  - Reject Null Hypothesis -> **improvement due to a statistically significant effect**
- Measure **Effect size** with *Rank-Biserial Correlation (r_rb)*
  - Negligible: $|r_rb| < 0.1$, Small: $0.1 < |r_rb| < 0.3$
  - Medium: $0.3 < |r_rb| < 0.5$, Large $|r_rb| > 0.5$

[1] Patel, Harsh, Bram Adams, and Ahmed E. Hassan. "Post deployment recycling of machine learning models: Don't Throw Away Your Old Models!." *Empirical Software Engineering* 29.4 (2024): 100.

Results – JIT vs TA-JIT

RQ1: Does leveraging ticket-level information increase the accuracy of JIT defect prediction?

- For every performance metric, the median score of TA-JIT **beats** the corresponding median of JIT,
  - **systematic shift in central tendency** in favour of the TA-JIT approach (Fig. 1)
- Rejected Null-Hypothesis in 31 out of 42 (*), indicating the TA-JIT **outperforms** JIT in the majority of metric-classifier-dataset combinations (Table 1)
- The “largest” effect size (i.e. Gain) is obtained using RF and LR classifiers (Table 1)

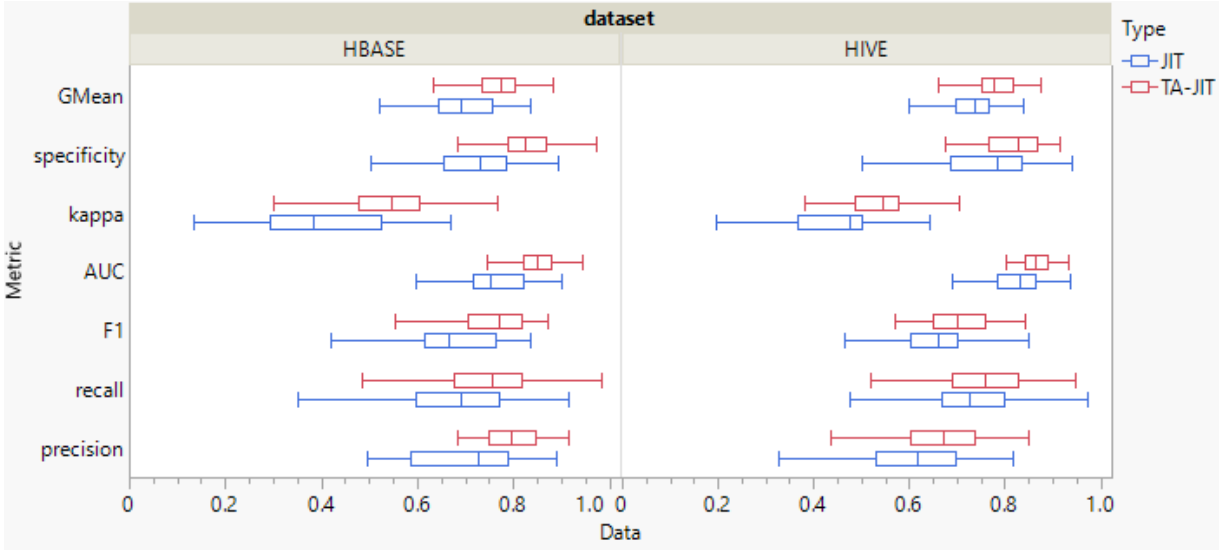


Fig.1 - Distribution of specific accuracy metrics over different windows and three classifiers of the JIT and TA-JIT approach

	HBASE			HIVE		
	LR	NN	RF	LR	NN	RF
AUC	Large*	Large*	Large*	Large*	Moderate	Large*
F1	Large*	Large	Large*	Large*	Large*	Large*
GMean	Large*	Small	Large*	Large*	Large*	Large*
Kappa	Large*	Small	Large*	Large*	Large*	Large*
Precision	Large*	Large	Large*	Large*	Large*	Large*
Recall	Small	Small	Large*	Moderate	Small	Large*
Specificity	Large*	Moderate	Large*	Large*	Large*	Large

Table 1 - Effect size (Large, Medium, Moderate, Small) and statistical significance difference (*) between TA-JIT and JIT for each combination of classifier and accuracy metric.

Results – JIT vs TA-JIT

RQ2: What is the predictive power of ticket-level features in JIT defect prediction?

- The percentage of selected JIT features varies between 30% and 60% (Fig.1)
  - It implies that in some of the windows, **most of the selected features are Ticket-Level!**
- Ticket-Level feature selection varies according to project under study
  - JIT family has a feature selected in 100% of windows (HBASE and HIVE)
  - *External-temperature* family has at least one feature selected in 82% of windows (HBASE)
  - *Internal-temperature* family has at least one feature selected in 80% of windows (HIVE)

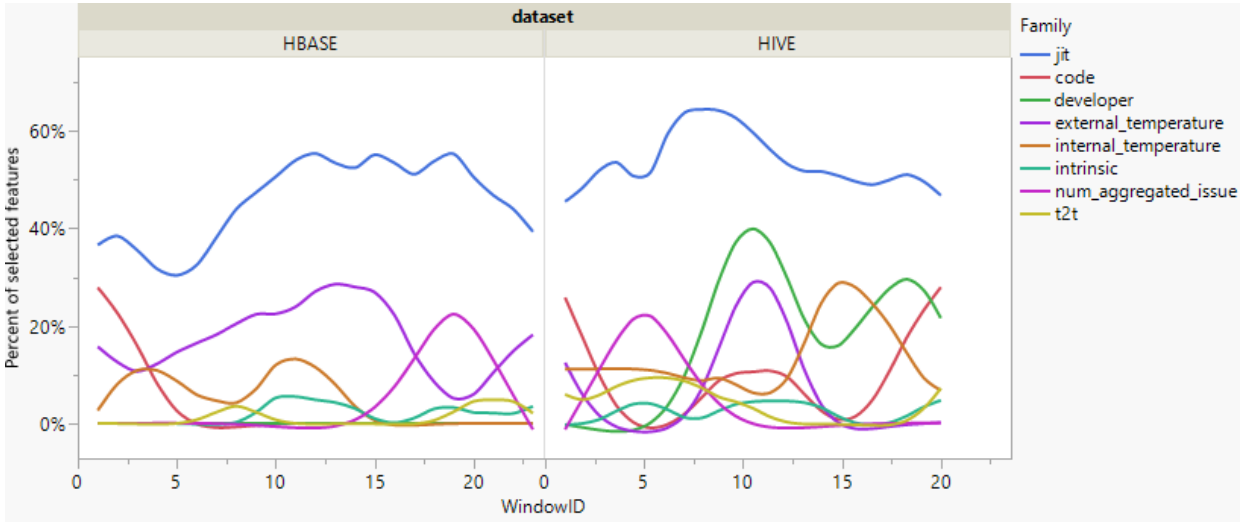


Figure 1 – Percentage of selected features in a specific window related to specific TL families

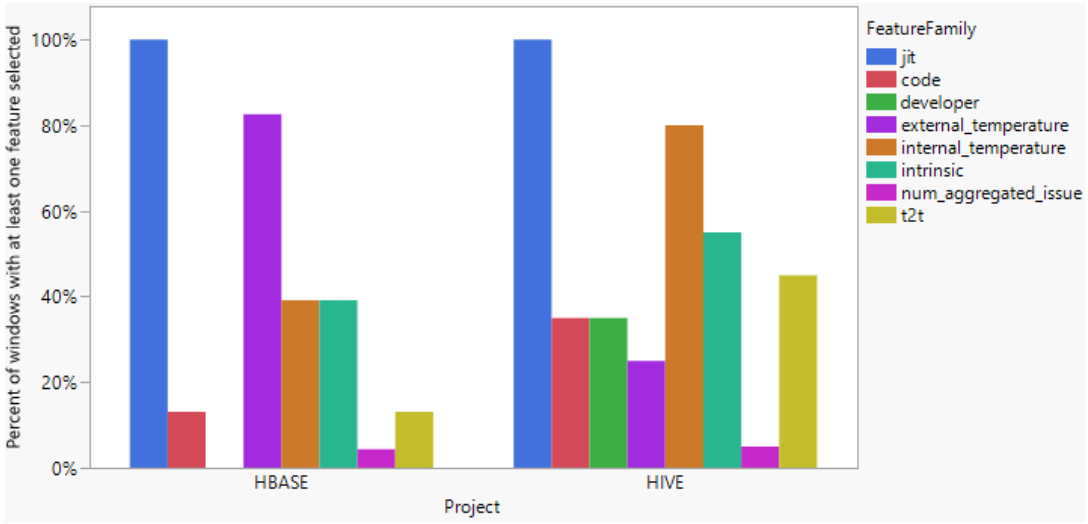


Figure 2 – Percentage of windows with at least one feature selected of specific families.

Discussion and Conclusion

Performance

- Ticket-Level information **supports** JIT Defect Prediction
- TA-JIT **outperform** JIT on all 42 metric-classifier-dataset combinations at the level of **median performance**
  - Superiority is statistically significant in 74% of the comparisons, with large effect size in 80% of the significant cases
- *AUC* and *F1* aggregate metrics show a large and significant improvement
- *Precision* (Positive-Predicted rate) increases more than *Recall* (True-Positive rate)
  - TA-JIT reduces “false alarms” while maintaining a high hit-rate
  - Confirmed also by improvement of *Specificity* (True Negative rate) and *G-Mean* ($\sqrt{Recall\ x\ Specificity}$)
  - TA-JIT Precision increases of 20% w.r.t JIT
    - e.g., given 400 commits in a two-week sprint, can avoid 80 false-positive alerts, saving about 35 engineer-hours per sprint.

Feature “dynamics”

- JIT remain the “prediction” core, TL features **complement** it
- Type and proportion of TL features fluctuate over time and projects
- No “silver-bullet” TL feature -> “adaptive” to context, valuable for companies



TOR VERGATA

UNIVERSITÀ DEGLI STUDI DI ROMA

[a];[b]; [c]